Name: Sri Ravela
GitHub Issue: https://github.com/cyber-shuttle/cybershuttle-ai4science-foundry/issues/24

Scientific Purpose and Selection:

FourCastNet is a global, data-driven weather forecasting model that predicts atmospheric variables (geopotential, temperature, wind, humidity, precipitation) at 0.25° resolution, trained on ERA5 reanalysis data. Traditional numerical weather prediction (NWP) solves physical PDEs on supercomputers, taking hours per forecast. In contrast, FourCastNet generates a week-long forecast in under two seconds on a single GPU (with comparable resolution). Generally, it is used by climate and atmospheric scientists, and also ML infrastructure teams building large-scale forecast and simulation pipelines.

My main motivation for selecting this model is because of the computer system work that I am interested in. My previous internship at a biotech-startup focusing on protein folding gave me interest in science-focused artificial intelligence work. I was interested in a broader area of assignments, like a weather and climate systems project with actual ML infrastructure concepts and core AI work involved. Concepts like Markov chains and probability also interest me, so I thought this project was the closest match to my interests and would give me the most in-depth practical application of how neural networks actually work and play a role in inference clusters/jobs. This project would also answer some technical questions I have, such as how FourCastNet's AFNO backbone replaces self-attention's pairwise token comparisons with a per-channel filter in the Fourier domain. This ends up being what lets it mix a 16,200-patch grid at close to linear cost instead of quadratic. Reproducing inference in this project means actually reasoning about how certain tradeoffs increase or decrease efficiency and memory footprint or rollout cost, which is the exact type of high-agency project that would help me become a better engineer.

Architecture and Contribution:

FourCastNet uses an Adaptive Fourier Neural Operator (AFNO) backbone with input atmospheric fields split into patches (as in a vision transformer) and then mixed globally per-channel using a learned filter applied in the Fourier domain (via FFT) instead of standard self-attention. This gives token mixing with near-linear complexity in the number of spatial locations, which is what makes high-resolution (720×1440 grid, 20 channels) global forecasting computationally tractable. A separate precipitation head (precip.ckpt) sits on top of the backbone specifically for precipitation forecasts. The paper actually claims it is a high-resolution, purely data-driven global forecasting model that matches or exceeds traditional NWP (ECMWF's IFS) skill for several variables at short-to-medium lead times, while being roughly four to five orders of magnitude faster at inference. That speed is what makes large-ensemble forecasting for

uncertainty quantification practical, something that's computationally out of reach for physics-based models.

Sources and Reproduction Target:

Paper: Pathak et al., "FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators," 2022. https://arxiv.org/abs/2202.11214

Official repository: https://github.com/NVlabs/FourCastNet

Weights/checkpoints: publicly released, backbone.ckpt and precip.ckpt (FCN_weights_v0) at https://portal.nersc.gov/project/m4134/FCN_weights_v0/, plus a stats_v0 folder (global_means.npy, global_stds.npy, time_means.npy, land_sea_mask.npy, latitude/longitude) required for de-normalizing model outputs.

Data: ERA5 reanalysis, published by ECMWF/Copernicus (https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels), also accessible via the same NERSC Globus endpoint as the weights. The repository README talks about a single ERA5 snapshot as sufficient to initialize a forecast.

Reproduction target: I want to reproduce the small-batch inference pipeline: load the pretrained backbone, initialize from an ERA5 analysis snapshot, run an autoregressive multi-step rollout (targeting a 5–7 day forecast), and compute root-mean-square error (RMSE) and anomaly correlation coefficient (ACC) against ERA5 ground truth for a small set of variables (e.g., Z500, T850, 10m wind), comparing the resulting skill-vs-lead-time curves qualitatively against the paper's Figure 3/4.

Licensing and Feasibility:

The official repository code is under a BSD 3-Clause license, which is fully permissive with no non-commercial or academic restriction. ERA5 itself is easily available from Copernicus/ECMWF and mirrored via NERSC Globus, although for the full archive, a Copernicus CDS registration is required. The pretrained checkpoints are public for research use, so there is no paid, geolocked, or restricted license in this reproducible pipeline.

Resource	Reasonable Estimate
GPU Memory	About 2-4 GB per forecast run based on released config (narrow 56-wide embedding instead of usual ViT's 768) on a single forward pass (batch size 1). This means the model is parameter-light with respect to embeddings size, so this is very manageable for the capacity of a single GPU on GT's PACE-ICE. 1 single-GPU job should be enough, but will be cross-referenced against peak memory in proper runs.
Runtime	The original paper suggests a week-long forecast in under 2 seconds on one GPU. Generally, a multi-seed/multi-initialization evaluation should stay under an hour of GPU clock runtime.
CPU/RAM	General Requirements; 8+ cores, 16–32 GB RAM for HDF5 I/O and pre/post-processing of gridded fields and weather data files (Standard Allocation)
Storage	Checkpoints (backbone.ckpt + precip.ckpt) and stats files need under 8GB, which is very manageable. However, some initialization snapshots may require 10+ GB, but do not use the full 5TB training set.
Data Transfer	A one-time Globus download generates weights, stats, and a scoped ERA5 subset with no repeated transfers. There are no large or repeated transfers after that.

Risks and Fallback Scope:

There are a few realistic ways that interfere with my intended inference requests. Globus and Copernicus CDS require account setup and variable download/load lengths, so there may be extra steps in this process that delay my jobs. Additionally, the generated skill scores depend on matching the paper's normalization and variable-selection pipelines precisely, so small differences or variations in methodology and results can change output. My plan is to use the released stats_v0 files as they come, so that I do not have to recompute the normalization

statistics myself and possibly mess up. Regardless, different GPU and cuFFT versions can also shift results from the paper's report, so instead of comparing exact digit percentiles, I will compare overall shape and ordering of my measured skill curve. One risk that I did not account for was the new PACE-ICE scheduling that was released by Dr. Lombardi of PACE-ICE, which now cap users at 50 queued or running GPU jobs and 32 concurrent GPU-hours for space concerns. Although my usage is measured to be considerably within these limits, a possible bigger multu-seed sweep later in my inference project may require deep sequencing instead of all at once. This is not a constraint as of now, but keeping it in mind can prevent delays and mess up GPU limitations on my account.

Fallback Scope: If limitations arise, I can scale down to a single variable (Z500) with a single initialization date, and a 3-day total rollout to limit cost computation. I can also only report RMSE and drop ACC and multi-variable comparison that may take up too much time. The project stays inference-based on the earlier pipeline process, but takes on a much narrower slice of how the statistics behind the data actually give implications on the results and full paper's benchmarks.